Homework 4

Algorithm Description:

We're given a table with 2^n - k test inputs (each one is an n-length boolean combo), and we know exactly k combos are missing, but the catch is we can't just read the table normally. The only way in is 'queryTable(row, var)', one bit at a time, and every call costs us. So the strategy is: don't brute-force the whole truth table, hunt only where the gaps are. We treat the space of all inputs like a binary decision tree over the variables. Any prefix of length d should contain exactly 2^{n-d} combinations if nothing were missing. We keep buckets of row indices that match a prefix, compare 'bucketSize' to 'expected = 2^(n-d)', and the moment a bucket is "full," we drop it and move on, no more queries wasted there. If it's not full, we query the next variable for the rows in that bucket once, split into false and true, and recurse. When d = n, a bucket with size 0 means that exact combination never showed up, it's missing, so we output it. The whole point is that we spend queries like a spotlight, drilling down only into prefixes that actually contain missing tests, instead of reading all n bits of everything.

Proof of Correctness:

We view each length-n boolean assignment as a path in a binary decision tree over the variables. A node at depth d corresponds to a prefix that fixes the first d variables, and a leaf at depth n corresponds to one exact input combination. The algorithm maintains, for each prefix P, a bucket B(P) containing row indices from the table. The algorithm splits buckets by querying the next variable, and prunes any bucket whose size matches the number of combinations that should exist under that prefix in a complete truth table.

Claim (Invariant): In every recursive call on prefix P, the bucket B(P) contains exactly the table rows whose assignments match prefix P.

Justification: Initially, the prefix is empty, so B(ε) contains all rows and the claim holds. For the recursive step, assume the claim holds for a prefix P at depth d. The algorithm queries queryTable(row, d) for each row in B(P) and places the row into either the false child bucket or the true child bucket. Each row is classified into exactly one child bucket based on its value for variable d, and every row in B(P) is classified. Therefore, the two child buckets contain exactly the rows matching the extended prefixes P0 and P1. Hence the invariant is preserved at every level.

For a prefix of length d, there are exactly 2^{n-d} possible full assignments consistent with that prefix (the remaining n-d variables are free). If $|B(P)| = 2^{n-d}$, then by the bucket invariant the table contains a row for every possible completion of P. Therefore, no combination consistent with P is missing, and it is safe to prune this bucket without losing any missing assignment.

When d=n, the prefix P is a complete assignment, so there is exactly one possible combination consistent with P. By the bucket invariant, |B(P)| equals the number of table rows that match that exact assignment. Thus:

- If |B(P)| = 0, then the assignment does not appear in the table and is missing, so the algorithm outputs it.
- If |B(P)| >= 1, then the assignment appears in the table and is not missing, so the algorithm does not output it.

The algorithm only outputs combinations that do not appear in the table (no false positives), and any missing combination cannot lie under a pruned prefix and will be reached at depth n and output (no false negatives).

∴ The algorithm outputs exactly the k missing combinations.

Time Complexity:

The cost of the algorithm is dominated by calls to queryTable(row, var). Each time we expand a prefix at depth d, we query variable d once for every row currently in that prefix-bucket, so the total number of queries is the sum of bucket sizes over all expanded nodes.

In the worst case, pruning provides little benefit and a row may be queried at up to n depths, giving $O((2^n - k) * n)$ total queries.

In the best case when k is small, most prefixes become "full" quickly and are pruned early, so we avoid reading all n variables for most rows.

For example when k=1 the total work is roughly $(2^n - 1) + (2^{n-1} - 1) + \ldots = O(2^n)$.

More generally, the algorithm smoothly interpolates between these extremes: it can be close to $O(2^n)$ when k is small, and approaches $O((2^n - k) * n)$ only when many tests are missing and pruning cannot cut off large regions.

Space Complexity:

At any split, a parent bucket of size s becomes two child buckets whose sizes sum to s. If we implement this as a depth-first recursion, we can discard the parent once it's split. So the peak bucket storage is on the order of "one level's worth of indices,": $S \in O(2^n - k)$

This is because at most we're holding row indices for the portion of the table we haven't pruned away yet, and in the worst case that's basically all rows.

- recursion stack / prefix tracking: O(n)
- output storage for missing combos: O(k * n)

So overall: $S \in O((2^n - k) + k * n)$ and the dominating term is almost always the $2^n - k$ row-index storage.